

Módulo 7: DevSecOps y Cierre

T19: CI/CD Seguro

Carlos Rodríguez Contreras · GitHub: karrocon

 *Secrets scanning, Dependabot y CodeQL en GitHub Actions*

Objetivos

- Configurar GitHub Actions con secrets seguros (no hardcodeados)
- Activar Dependabot para alertas de seguridad automáticas
- Integrar SAST con CodeQL como security gate en el pipeline
- Entender el concepto de DevSecOps: shift-left security

DevSecOps – Shift-Left Security

Tradicional: Dev → Ops → Sec (seguridad al final)
DevSecOps: Dev+Sec → CI/CD+Sec → Deploy+Sec (seguridad desde el inicio)

Shift-Left = mover la seguridad lo más a la izquierda posible en el ciclo de vida:

Fase	Security Check
Código	Pre-commit hooks (git-secrets), IDE linters
PR	SAST (CodeQL), secrets scan, <code>npm audit</code>
Build	Image scan (Trivy), DAST básico
Deploy	Configuración segura (IaC scan), HTTPS
Runtime	WAF, monitoring, alertas

GitHub Secrets — no hardcodear en workflows

```
# ❌ NUNCA hardcodear secretos en el workflow
env:
  API_KEY: sk-proj-abc123... # ← visible en el repo

# ✅ Usar GitHub Secrets
env:
  API_KEY: ${ secrets.API_KEY } # ← cifrado, nunca visible en logs
```

⚠️ GitHub enmascara secretos en logs, pero cuidado con `base64` u otras transformaciones que pueden exponerlos.

Workflow de seguridad con GitHub Actions

```
# .github/workflows/security.yml
name: Security Checks
on: [push, pull_request]

jobs:
  audit:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: npm ci
      - run: npm audit --audit-level=high

  codeql:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: github/codeql-action/init@v3
        with:
          languages: javascript-typescript
      - uses: github/codeql-action/analyze@v3

  secrets-scan:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: gitleaks/gitleaks-action@v2
```

Dependabot – actualizaciones automáticas

```
# .github/dependabot.yml
version: 2
updates:
  - package-ecosystem: "npm"
    directory: "/VulnApp"
    schedule:
      interval: "weekly"
    open-pull-requests-limit: 10
    labels:
      - "dependencies"
      - "security"
```

Resultado: cada semana, Dependabot abre PRs automáticos para dependencias con CVEs.

Combinar con **tests automáticos** = merge con confianza.

SAST vs DAST

Tipo	Herramienta	Cuándo	Qué detecta
SAST (estático)	CodeQL, Semgrep	En cada PR	Patrones de código inseguro
DAST (dinámico)	OWASP ZAP, Burp	Post-deploy	Vulnerabilidades en runtime
SCA (composición)	npm audit, Snyk	En cada build	CVEs en dependencias
Secrets	gitleaks, truffleHog	Pre-commit + CI	Secretos filtrados

✓ **Pipeline ideal:** SAST + SCA en PR → bloquean merge si hay HIGH/CRITICAL.

Branch Protection – gates obligatorios

Configuración en GitHub → Settings → Branches → main

Branch protection rules:

- ✓ Require pull request before merging
- ✓ Require status checks to pass (security workflow)
- ✓ Require review from CODEOWNERS
- ✓ Do not allow bypassing the above settings
- ✓ Require signed commits

Gate	Bloquea merge si...
<code>npm audit</code>	Hay vulnerabilidades HIGH/CRITICAL
CodeQL	Detecta patrones inseguros
gitleaks	Encuentra secretos en el diff
Tests	Fallan tests unitarios o de integración

💡 Nadie (ni admins) debería poder mergear a `main` sin pasar los security gates.

Environment Protection Rules

```
# GitHub → Settings → Environments → production
```

```
Environment: production
```

- ✓ Required reviewers: **@security-team**
- ✓ Wait timer: **5 minutes** (cool-down period)
- ✓ Deployment branches: **only "main"**

```
Secrets: (scoped to this environment only)
```

- **AZURE_CREDENTIALS**
- **PROD_DATABASE_URL**

Beneficio: los secretos de producción solo están disponibles en workflows que despliegan a ese environment, y requieren aprobación humana.

Lab T19: Pipeline con security gates

Objetivo: crear un workflow de GitHub Actions con checks de seguridad

Setup: repositorio en GitHub

1. Crea `.github/workflows/security.yml`
2. Añade un job que ejecute `npm audit --audit-level=high`
3. Añade un job con CodeQL para análisis estático de seguridad
4. Activa Dependabot: crea `.github/dependabot.yml` con config para npm